National University of Computer and Emerging Sciences

AAgentic AI
(AI4015)

Assignment Beta

Date:19/10/2026

Course Instructor(s)

Dr. Muhammad Farrukh Shahid

Submission Deadline 8th November 2026

Total Marks: 100

Total Questions:5

Semester: FALL 2026

Campus: Karachi

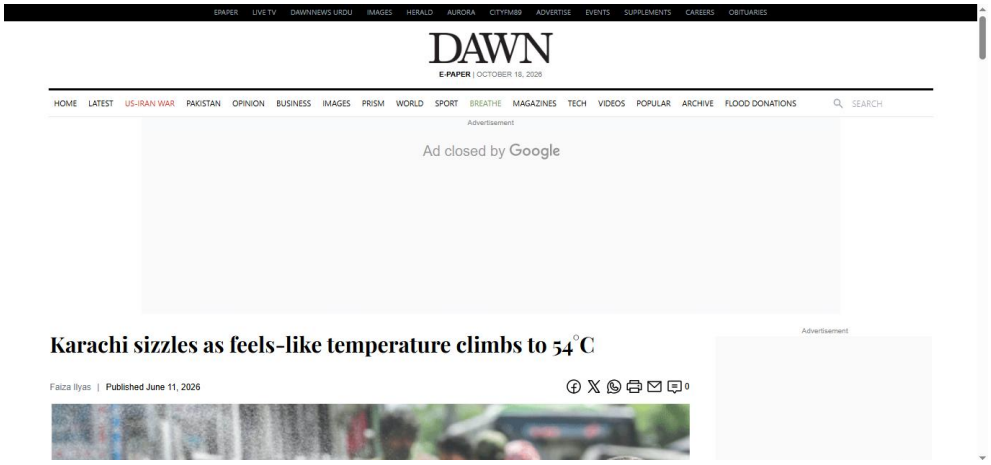
Dept: School of Computing

Instructions: Check submission requirments after each question.

CLO #2:

Question no 1

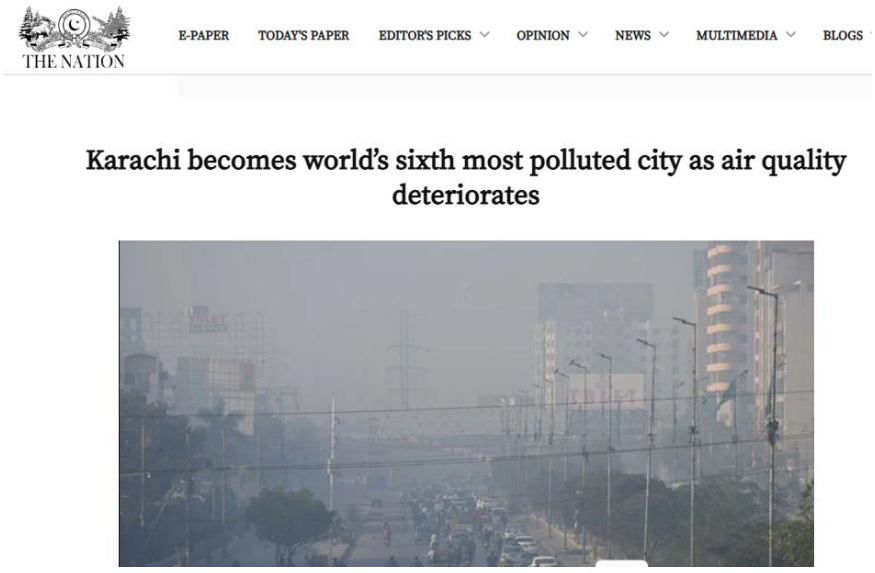
Anyone who has experienced a summer season in Karachi,Pakistan knows how difficult and problematic life can become. Early in the morning, a thick grey haze often hangs over the city's industrial areas and highly dense areas. By May and June, rising temperatures and intense and scorching heatwaves make even a routine journey to work or school/ colleges/ universities exhausting and, at times, dangerous and threatening. The situation is particularly concerning in densely populated neighbour-hoods, where many residents have limited access to timely information about air quality and extreme heat. As a result, citizens often step outside without realizing the health risks they may face and get ill due to this situation. Some references:



Source https://www.dawn.com/news/2006711



Source https://tribune.com.pk/story/2542426/karachi-scorches-as-met-dept-warns-of-heatwave-across-sindh



Source https://www.nation.com.pk/09-Oct-2025/karachi-becomes-world-s-sixth-polluted-city-air-quality-deteriorates

A tech company wants to develop a system using some AI-enabled solutions for the citizen of Karachi related to climate and heatwave information and awareness. It is building **SaafHawa**, an agentic AI assistant that Karachi citizens can message on Whats App for advice on air quality and heat waves, temperature rise situation and get health related advice. Typical questions include:

"Aj din 12 bajy temperature kitna rise hoga q k bachon ki school 12 bajy off hojaye ga?"

"Is it safe to go for an evening jog by the sea today?"

To answer a questions asked by the citizens, the agent must do the followings:

1. Identify the area type, the time of day and the person at risk, such as a child, an elderly person, an asthma patient or an outdoor labourer.
2. Call an **air-quality tool** that returns real PM2.5 and PM10 readings from an open source such as OpenAQ.
3. Call a **weather tool** that returns temperature, humidity and wind from a source such as Open-Meteo.
4. Call a **heat-index tool** that computes the heat index from temperature and humidity.
5. Retrieve relevant guidance from a small **knowledge base** of official health and heatwave advisories.
6. Reply briefly and practically, in the language and script the user wrote in.

Each morning, the agent also prepares a **pollution summary** for the municipal officers. This raises the stakes considerably. An incorrect figure in that summary does not mislead a single citizen; it reaches people who make decisions on behalf of thousands. Before any code is written, the Tech company technical lead puts one question to you:

"Should we pay for a closed-source API, or host an open-weight model on our own servers?

I have seen enough leader-board screenshots. Show me what actually works for our traffic, our languages and our budget."

Your task is to answer that question with evidence you have gathered and numbers you have calculated yourself.

Parameters

Let **R** be the last two digits of your roll number, and let **D** be the sum of all the digits in your roll number, taken mod 10 (ignore any letters or hyphens). The parameters are derived from these values, as shown in the table below.

Parameter	Formula
Normal daily queries, Q	1,500 + 25 × R
LLM calls per query (first attempt), N	5 + (R mod 4)
Tool-call failure probability per call, p	0.04 + 0.01 × D
Heatwave days per month, H	4 + (R mod 4)
Monthly inference budget, B (USD)	1,200 + 10 × R
End-to-end latency target, T (seconds)	10 - (R mod 3)

Aassumptions

- **Token growth:** context lengthens at every step as history and tool outputs accumulate. For call k (k = 1, 2, …, N), input tokens = 2,000 + 800 × (k − 1). Each call produces 350 output tokens, except the final call, which produces 600.
- **Tool calls:** calls 1 to N − 1 each produce exactly one tool call, which is executed before the next LLM call begins. Call N produces the final answer and makes no tool call.
- **Cache-able prefix:** the first 1,500 input tokens of every call consist of the system prompt and tool schemas, and are identical across all calls.
- **Retries:** each of calls 1 to N − 1 produces an invalid tool call with probability p. When this happens, the agent repeats that call once with the same input, and the retry always succeeds. Call N is never retried.
- **Reasoning mode:** run every model in its non-reasoning or lowest-reasoning setting, and state the setting you used. If a model still generates reasoning tokens in that setting, add them to its output tokens for both cost and latency, using an average reported by the provider or a cited benchmark source.
- **Heatwave traffic:** on each of the H heatwave days, traffic rises to 5 × Q, and 35% of that day's queries arrive between 12:00 and 14:00. Every other day carries Q queries. Assume a 30-day month.

Mention R, D and every derived value on the first page of your submission.

Section 1:

Select **at least six closed-source models** and **at least six open-weight models**, all current at the time of submission. Each side must include models from at least four different providers. For every model, report:

1. Parameter count, written as "not disclosed" where the provider does not publish it; for mixture-of-experts models, give both **total** and **active** parameters.
2. Context window and maximum output tokens.
3. Price per million input, output and **cached** input tokens; for open-weight models, quote the lowest price offered by a public inference provider and name that provider.
4. Rate limits, in requests per minute and tokens per minute, for the tier available to a new paid account.
5. Reported time to first token (TTFT) and output throughput in tokens per second.
6. Scores on at least **two public benchmarks**, one of which measures tool use or function calling.
7. Whether native tool calling and structured JSON output are supported, and which reasoning settings the model offers.
8. Licence, and whether commercial self-hosting is permitted.

Present the results in a **single table**, citing a source and access date for every figure. Conclude the section with a paragraph that names two values you could not verify and explains how that uncertainty affects your later analysis.

Section 2:

Show every step of your working. No marsk on direct answer.

1. **Token volume.** Calculate the expected input and output tokens per query, including retries. Then compute monthly totals using your Q and H.
2. **API cost.** For all twelve or more models, compute the monthly cost with and without prompt caching on the 1,500-token prefix. Include any premium the provider charges for writing to the cache, and state its minimum cacheable prefix length and cache lifetime. If 1,500 tokens is below a provider's minimum, assume no caching for that provider. Identify which models remain within B in each case.
3. **Peak load and rate limits.** Compute peak queries per minute and tokens per minute during the heatwave window.
4. **Self-hosting memory.** Choose one open-weight model from **Part A** that uses standard full attention, either multi-head or grouped-query. If you choose a model with sliding-window, hybrid linear or latent attention instead, adapt the KV-cache formula below to its architecture and justify the change. From the model's config.json on Hugging Face, record the number of layers, KV heads and head dimension, and include a screenshot of these values. If head_dim is not listed, use hidden_size ÷ num_attention_heads. Then calculate:
    1. Weight memory at FP16, 8-bit and 4-bit precision;
    2. KV-cache memory per sequence at the longest context in your agent loop, using KV bytes = 2 × layers × kv_heads × head_dim × sequence_length × bytes_per_value, with 2 bytes per value for an FP16 cache;
    3. The maximum number of concurrent sequences on one real GPU of your choice, assuming 4-bit weights and an FP16 KV cache, and reserving 10% of the GPU's memory for activation and runtime overhead; and
    4. The number of GPUs required to serve the concurrent sessions calculated in item 3.
    5. Break-even analysi**s.** Using a real cloud GPU price, calculate the monthly cost of self-hosting with the GPU count from item 4, assuming every GPU runs for the full month whether busy or idle. Because one GPU can serve only a limited number of concurrent sequences, model self-hosting cost as a step function of daily query volume: as volume grows, add GPUs according to the per-GPU capacity you calculated in item 4. Then find the daily query volume, Q*, at which self-hosting becomes cheaper than the cheaper of your two candidates from item 6 that meets T. If neither candidate meets T after your architectural change, use the one with the lower latency. Plot monthly cost against daily query volume for both options, and assess whether Q* is a realistic volume for an Tech of this size.

6. Latency budget**.** For your two strongest candidates, estimate the end-to-end latency of one query. Calls run sequentially, each call is longer than the one before it, and each tool call adds a delay equal to the median tool latency you measure yourself. To obtain it, call the air-quality API and the weather API at least 20 times each, spread over at least two different days, time the heat-index calculation and knowledge-base lookup on your own machine, and use the median of all these measurements as the delay for every tool call. Submit the raw timing log. Include the expected delay from retries. If a candidate exceeds T, propose one specific architectural change and recalculate. Suitable changes include routing simple steps to a smaller model, running independent tool calls in parallel and streaming the final response.

Submission

- Pdf/ Word containing your parameters page, comparison table, calculations and cost plot;
- The spreadsheet or notebook in which you performed your calculations;
- Your raw tool-latency timing log; and
- A short declaration of any AI tools you used and the purpose of each.

CLO #3:

Question no 2

Consider an emerging e-commerce startup hires AI engineer. The CTO introduces AI to a project the company has been struggling with: developing an AI-powered customer support chatbot that can understand customer requests and respond accurately.

The idea seems straightforward. When a customer asks something like, "**Where is my order**? Customer placed it last Tuesday," the chatbot should understand the request, identify the appropriate function, and generate a valid JSON response that the backend system can execute. The company has already defined four functions: check_order_status, initiate_refund, track_shipment, and escalate_to_human.

However, the existing chatbot is far from reliable. Although the team has selected a small open-source language model that communicates reasonably well with customers, it struggles when asked to use tools. Sometimes it adds unnecessary explanations before or after the JSON output. At other times, it generates function names that do not exist in the system. More concerning, it occasionally selects the wrong function, such as initiating a refund when a customer has only asked about the expected delivery date. These mistakes may seem minor during testing, but in a real e-commerce environment, they could lead to incorrect transactions, dissatisfied customers, and unnecessary operational costs.

The CTO wants these problems resolved, but there is an important limitation: the startup has access to only one GPU. Fully fine-tuning a large language model would require more computational resources than the company can afford. AI engineer therefore need an approach that improves the model's ability to select and call tools without placing excessive demands on the available hardware.

This is where parameter-efficient fine-tuning becomes relevant. A Low-Rank Adaptation (LoRA), a technique that allows language models to be adapted to specific tasks by training a relatively small number of additional parameters rather than updating the entire model.

Original paper of LORA can be accessed from https://arxiv.org/abs/2106.09685

AI engineer explores the available methods, however, he discovers that LoRA is not the only option. Other approaches, including Quantized LoRA (QLoRA), Weight-Decomposed Low-Rank Adaptation (DoRA), and Adaptive LoRA (AdaLoRA), offer different ways to balance model performance, memory consumption, and training efficiency. Rather than selecting a technique based on popularity or published claims alone, AI engineer decides to conduct a systematic comparison. Using the same base model, training dataset, and evaluation criteria wherever possible, examine how each method performs on the customer support task.

The objective is to identify a fine-tuning approach that is accurate, reliable, and practical for deployment. Ultimately, the best method will not necessarily be the most sophisticated one, but the one that delivers consistent results while making efficient use of the resources available to the company.

Note: Your evaluation will consider not only whether the model selects the correct function, but also whether it generates valid JSON, avoids unauthorized or unnecessary actions, and operates within the startup's computational constraints.

Section 1:

Consider a frozen weight matrix W_0 of size 4096×4096, and you attach a LoRA adapter with rank $r = 16$.

(a) How many trainable parameters does the adapter add? Show your working.

(b) What percentage is that of the layer's original parameter count?

(c) In LoRA, matrix B starts at zero and matrix A starts random. Why? What would go wrong on the very first training step if both were random?

(d) **The "why does this exist?" question.** For each method below, provide what problem it solves that plain LoRA doesn't.

- **QLoRA** (mention NF4 quantization and paged optimizers)
- **DoRA** (mention how it splits a weight into magnitude and direction)
- **AdaLoRA** (mention how it decides where rank is actually needed)

Section 2:

Pick Qwen2.5-1.5B-Instruct (or Llama-3.2-1B-Instruct if you'd rather) as your base model. Take a 2,000-example slice of a function-calling dataset such as glaive-function-calling-v2, and keep 200 of those aside as your test set. Don't peek at them while training. Use the Hugging Face PEFT library.

You'll fine-tune the same model four times. The only thing that changes between runs is the adaptation method. Same data, same seed, same number of epochs, same learning-rate schedule, same target modules. If you change two things at once, you won't know which one made the difference, and neither will I.

Run	Method	Base precision	Rank
1	LoRA	bf16 / fp16	r = 8
2	QLoRA	4-bit NF4	r = 8
3	DoRA	bf16 / fp16	r = 8
4	AdaLoRA	bf16 / fp16	start r = 12, target r = 8

Also run the original base model with no fine-tuning at all. This is your "before" picture, and without it the "after" numbers don't mean much. That gives you five rows in your results.

For every run, record:

1. How many parameters were trainable, and what percentage of the whole model that is
2. Peak GPU memory during training (torch.cuda.max_memory_allocated() will tell you)
3. How long training took, in real minutes
4. **JSON validity:** out of 200 test outputs, how many actually parse with json.loads()?
5. **Tool accuracy:** how often did the model pick the correct function?
6. **Full match:** how often did it get both the function and every argument exactly right?

Section 3: Is a bigger rank actually better?

Take whichever method came out on top in Part B and train it three more times at r = 4, r = 16 and r = 32. Plot tool accuracy against rank.

Then answer this honestly: does doubling the rank really buy you anything here, or are you just spending more memory for the same result? There's no "correct" answer I'm looking for. I just want your conclusion to follow from your graph, not from what you expected to see.

Section 4:

Write a short report, four pages at most . Cover these four things:

5. **Your call.** If you had to ship one of these to production next Monday, which one would it be, and why? Weigh accuracy, memory, training time, and inference speed. A hint worth thinking about: QLoRA and DoRA don't behave the same way once you merge the adapter back into the base weights.
6. **Where it still breaks.** Pick five test examples your best model still got wrong. For each one, show the customer's message, what the correct call should have been, and what the model actually produced. Then label the mistake: wrong tool, made-up argument, broken JSON, refused to call a tool, or something else. Do you see a pattern?
7. **The agentic angle.** In a real multi-agent setup, you might have a router agent, a refund agent and a shipping agent, each needing slightly different behaviour. Explain how LoRA adapters make this much cheaper than training and hosting three separate models. Then name one real risk of swapping adapters in and out while the system is live.
8. **What surprised you?** Be specific. "LoRA worked well" doesn't count. Highlight where something didn't go the way you thought it would.

Submission

- Pdf/ Word containing your parameters page, comparison table, calculations and cost plot.
- The spreadsheet or notebook in which you performed your calculations.
- Notebook save with the name Assignment_Beta_Question_no_2_Student_ID

Question no 3

Under the knowledge distillation scenario, a large teacher model and a smaller student model are both predicting the next word after this prompt:

"The patient was admitted to the ___"

Assume the vocabulary contains only four candidate words:

Index	Word
1	hospital
2	clinic
3	ward
4	school

The correct next word is **hospital**, hence the one-hot label is **y = [1, 0, 0, 0]**.

The two models produce the following raw scores (logits):

- Teacher logits: $\mathbf{z_t}$ **= [5.0, 3.0, 2.0, 0.0]**
- Student logits: $\mathbf{z_s}$ **= [3.0, 2.5, 1.0, 0.5]**

Training uses a distillation temperature of **T = 2** and a mixing weight of **α = 0.5**. The training loss is:

$$L = \alpha \cdot T^2 \cdot KL(p_T \mid q_T) + (1 - \alpha) \cdot CE(y, q_1)$$

Here p_T is the teacher's softened distribution, q_T is the student's softened distribution, and q_1 is the student's ordinary distribution at T = 1. Use the natural logarithm throughout the calculation.

Calculate the following:

(a) The teacher's and the student's probabilities at T = 1.

(b) The softened probabilities of both models at T = 2.

(c) The KL divergence between the softened distributions.

(d) The hard-label cross-entropy loss.

(e) The total distillation loss.

(f) The gradient of the loss with respect to the student's logits.

(g) The student's logits after one gradient-descent step with learning rate η = 0.5, and the loss after that step.

Question no 4

An **AI team** uses a large language model (LLM) to classify **300 customer reviews** into three sentiment classes: **Positive, Negative, Neutral**. Each review has a human-annotated gold label.

Stage 1 Zero-shot baseline. The LLM is prompted with only the label names. Its confusion matrix (rows = actual, columns = predicted) is:

Actual \ Predicted	Positive	Negative	Neutral	Row total
Positive	100	5	15	120
Negative	8	80	12	100
Neutral	20	10	50	80
Column total	128	95	77	300

Stage 2 Improved prompt. Based on the Stage 1 error analysis, the prompt is revised. It now includes (i) an explicit definition of Neutral ("factual, mixed, or no clear opinion") and (ii) six few-shot examples, two per class, including borderline mixed reviews. The same 300 reviews give:

Actual \ Predicted	Positive	Negative	Neutral	Row total
Positive	106	4	10	120
Negative	6	84	10	100
Neutral	9	7	64	80
Column total	121	95	84	300

Generated justification. The Stage 2 prompt also asks the LLM to justify each label in one sentence. For one mixed review (gold label Neutral), the human-written reference and the LLM's output are (lower-cased, punctuation removed):

- **Reference:** the delivery was late but the product quality is good
- **Candidate (LLM):** the product quality is good but delivery was slow

The probabilities the LLM assigned to each of its 9 generated tokens, in order, were: 0.60, 0.40, 0.50, 0.80, 0.70, 0.30, 0.25, 0.50, 0.20.

Answer the following:

1. Overall accuracy of the zero-shot LLM, with a 95% confidence interval.
2. Precision, recall and F1 for each class (zero-shot).
3. Macro, weighted- and micro-averaged F1 (zero-shot).
4. Interpret Stage 1: where does the LLM fail, and what changes would you recommend?
5. Re-evaluate the Stage 2 model with the metrics from (1)-(3) and compare with Stage 1.

6. The few-shot prompt was run 5 times with different example orderings, giving accuracies 0.847, 0.840, 0.853, 0.837, 0.850. Report mean ± standard deviation and comment on prompt stability.
7. Evaluate the generated justification against the reference using BLEU-4, ROUGE-1, ROUGE-2, ROUGE-L and METEOR.
8. Compute the perplexity of the generated justification and interpret it.

Submission

- Pdf/ Word containing your solution.
- The spreadsheet or notebook in which you performed your calculations.
- Notebook save with the name Assignment_Beta_Question_no_4_Student_ID

CLO #3:

Question no 5

Refer the paper **A Comparative Evaluation of RAG Architectures for Cross-Domain LLM Applications: Design, Implementation, and Assessment**

Write a summary of the paper of two pages.

Submission

- Pdf/ Word summary of the paper of two pages.

Good luck